

Attention is all you need

≡ 분야

DL

🔑 3줄 요약

1. 기존 번역 모델은 문장을 한 단어씩 순서대로 처리하는 RNN(순환신경망) 구조라 **느리고**, 문장이 길어지면 앞부분 정보를 잊어버리는 문제가 있었다.
2. 저자들은 순환 구조를 완전히 버리고, "**어텐션(Attention)**"이라는 **메커니즘**만으로 문장 전체를 한 번에 처리하는 **Transformer**를 제안했다.
3. 결과적으로 번역 품질은 더 좋아졌고, 학습 속도는 훨씬 빨라졌다. 이 구조가 이후 ChatGPT 같은 거대 언어모델들의 기본 골격이 되었다.

🔑 3줄 요약

1. 이 논문이 풀려는 문제

2. 기존 방법들의 한계

3. 핵심 아이디어: Transformer 구조

3.1 큰 그림: 인코더-디코더

3.2 어텐션(Attention)이란 무엇인가

3.3 멀티헤드 어텐션(Multi-Head Attention)

3.4 Position-wise 피드포워드 네트워크

3.5 임베딩과 소프트맥스

3.6 위치 인코딩(Positional Encoding)

4. 왜 굳이 셀프 어텐션인가

5. 학습은 어떻게 했나

6. 결과: 얼마나 잘했나

7. 결론 & 이 논문이 왜 중요한가

💡 핵심 용어 한눈에 정리

1. 이 논문이 풀려는 문제

이 논문은 기계 번역(영어 → 독일어, 영어 → 프랑스어) 같은 "한 문장을 입력받아 다른 문장을 출력하는" 작업(=시퀀스 변환, sequence transduction)을 다룹니다.

당시 가장 좋은 성능을 내던 모델들은 **RNN(순환신경망)** 계열, 특히 LSTM 기반 인코더-디코더 구조였습니다. 문장을 이해하는 인코더(encoder)와, 그걸 바탕으로 번역문을 만들어내는 디코더(decoder)로 구성되고, 둘 사이를 어텐션으로 연결하는 방식이었죠.

비유: RNN은 문장을 한 단어씩 순서대로 읽어나가는 사람과 비슷합니다. "나는 어제 학교에" 까지 읽고 나서야 "갔다"를 처리할 수 있어요. 한 단어를 처리하려면 그 앞 단어까지의 정보가 먼저 계산되어 있어야 합니다.

2. 기존 방법들의 한계

기존에는 RNN, LSTM, Gated recurrent neural network가 사용.

$h_t = h_{t-1} + f(t)$ 로 hidden state가 정의되기에 RNN 방식에는 두 가지 큰 문제가 있었습니다.

- **병렬 처리가 안 됨:** 단어를 순서대로 하나씩 처리해야 하므로, 문장이 길어질수록 계산 시간이 그만큼 늘어나고 GPU를 여러 개 써서 한꺼번에 계산하는 게 어렵습니다.
- **장거리 의존성 문제:** 문장 앞부분의 정보가 뒤로 갈수록 희미해집니다. "그 책은 ... (수십 단어 뒤) ... 그것을"에서 "그것"이 "그 책"을 가리킨다는 걸 파악하려면, 그 사이 모든 단어를 순서대로 거쳐야 해서 정보가 유실되기 쉬워요.

이후 합성곱 신경망(CNN)을 쓰는 시도(ConvS2S, ByteNet 등)도 있었지만, 이 역시 멀리 떨어진 단어끼리 관계를 파악하려면 여러 층을 거쳐야 한다는 한계가 있었습니다.

→ averaging attention-weighted positions을 쓰면 상수번의 operation으로 처리할 수 있다!

저자들의 발상: "그럼 순서대로 처리하는 구조 자체를 버리고, 문장 속 모든 단어가 서로를 한 번에 동시에 쳐다보게(attend) 만들면 어떨까?" — 이게 Transformer의 출발점입니다.

Self-attention(intra-attention)으로 reading comprehension, abstractive summarization, texture entailment, learning task-independent sentence representation을 할 수 있다.

3. 핵심 아이디어: Transformer 구조

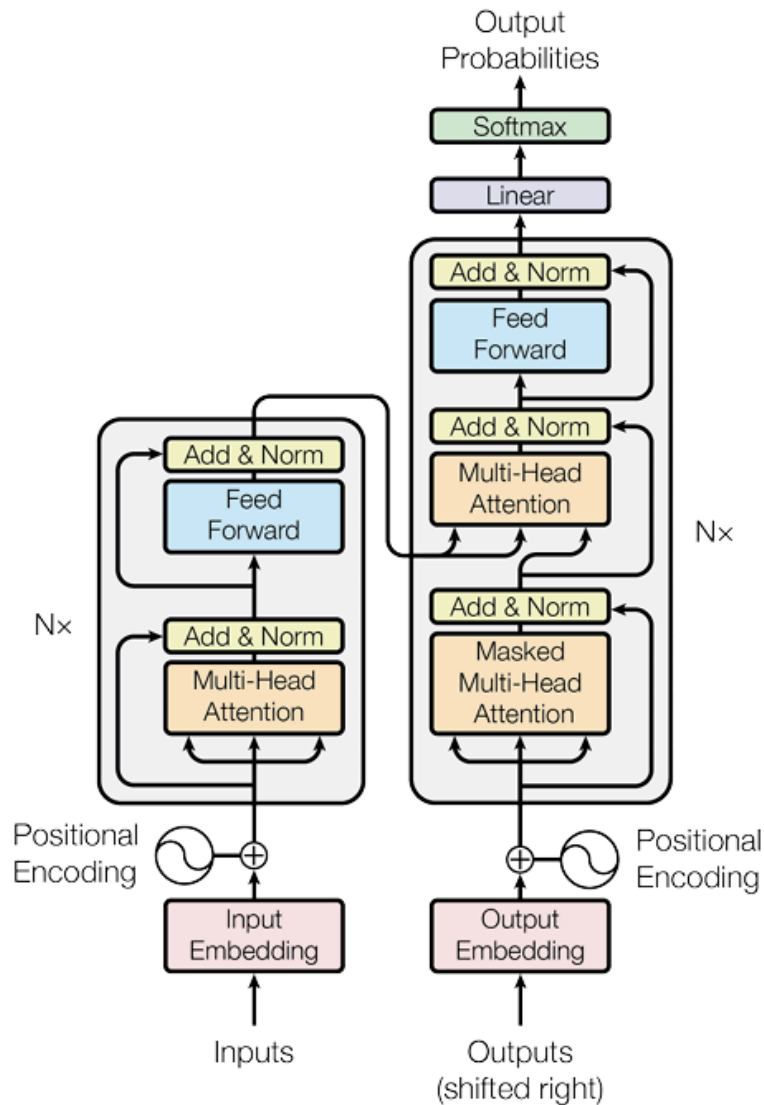


Figure 1: The Transformer - model architecture.

input : $\mathbf{z} = (z_1, \dots, z_n)$ 한 문장을 어절 단위로 끊은게 각각 원소

output : $\mathbf{y} = (y_1, \dots, y_m)$ task목적에 맞는 output문장을 어절단위로 끊은 것.

auto regressive 모델로 원소 하나하나씩 출력되며 input과 앞의 출력된 output원소들을 사용하여 다음 output원소를 예측.

3.1 큰 그림: 인코더-디코더

Transformer도 기존처럼 **인코더(입력 문장을 이해) + 디코더(출력 문장을 생성)** 구조입니다. 다만 각각 RNN이 아니라, **동일한 블록을 6개씩 쌓은 구조**입니다.

- **인코더:** 6개 층. 각 층은 "셀프 어텐션" + "Fully Connected 피드포워드 네트워크" 두 부분으로 구성

- **디코더**: 6개 층. 인코더와 비슷하지만, "인코더 결과를 참고하는 어텐션"이 하나 더 있고, 미래 단어를 미리 보지 못하도록 막는 장치(마스킹)가 추가됨

각 층 사이에는 ****잔차 연결(residual connection)****과 그 후 layer normalization이 있어서(ResNet과 같은 아이디어), 입력 정보가 손실 없이 다음 층까지 잘 전달되도록 도와줍니다.

3.2 어텐션(Attention)이란 무엇인가

어텐션의 핵심 아이디어를 비유로 설명하면:

문장 속 한 단어가 "지금 이 단어를 이해하려면 문장의 어느 부분을 더 신경 써서 봐야 할까?"를 스스로 판단해서, 관련 있는 단어들의 정보를 가중치를 두어 모아오는 것

이걸 수학적으로는 **Query(질문), Key(색인), Value(내용물)** 세 가지로 표현합니다.

- **Query**: "나는 지금 이런 정보가 필요해"
- **Key**: 다른 단어들이 "나는 이런 정보를 갖고 있어"라고 붙여둔 이름표
- **Value**: 실제로 갖고 있는 정보 내용

즉 Query는 질문이고 {Key : Value}가 응답인듯.

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

내 Query와 각 단어의 Key가 얼마나 잘 맞는지 (내적, dot product or additive attention을 통해서) 계산해서 점수를 매기고, 이 점수를 softmax로 확률처럼 정규화한 다음, 그 비율대로 각 단어의 Value를 섞어서 최종 결과를 만듭니다. 이걸 논문에서는 **Scaled Dot-Product Attention**이라 부릅니다(값이 너무 커지지 않도록 나눠주는 스케일링만 추가됨 → 너무커지면 softmax function 적용했을때 그라디언트가 너무 작아지니까).

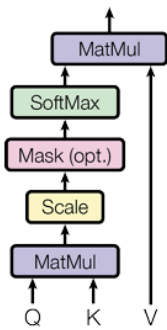
→ Additive attention : 1개 은닉층의 feed forward network로 compatibility function을 계산함.

→ additive attention보다 내적을 쓰는 이유 : 연산이 더 빠르고 공간효율적이다.

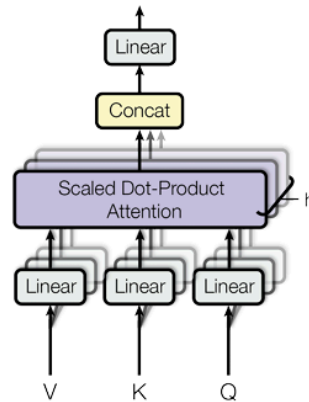
→ d_k가 작으면 두 연산은 거의 동일, but 커지면 내적이 유리.

도서관 비유: Query는 "내가 찾는 책의 주제", Key는 "서가에 붙은 분류 라벨", Value는 "실제 책 내용"이라고 생각하면, 라벨(Key)과 내 관심사(Query)가 잘 맞는 서가일수록 그 책 내용(Value)을 더 많이 참고하는 셈입니다.

Scaled Dot-Product Attention



Multi-Head Attention



3.3 멀티헤드 어텐션(Multi-Head Attention)

한 번만 어텐션을 계산하지 말고, 서로 다른 8개의 "시선(head)"으로 동시에 어텐션을 계산한 뒤 합칩니다. 이렇게 하면 어떤 헤드는 문법 구조를, 어떤 헤드는 의미적 연관성을 잡아내는 식으로 다양한 관점에서 문장을 이해할 수 있습니다.

비유: 같은 그림을 한 사람이 아니라 8명의 전문가(색채 전문가, 구도 전문가, 인물 전문가...)가 각자의 관점에서 동시에 분석한 뒤 의견을 종합하는 것과 비슷합니다.

각각의 head는 차원이 줄어들었기 때문에 연산량은 기존과 비슷.

3.4 Position-wise 피드포워드 네트워크

$$FFN(x) = \max(0, xW1 + b1) W2 + b2$$

어텐션으로 정보를 모은 뒤, 각 단어 위치마다 독립적으로 작은 2층짜리 신경망을 하나 더 통과시켜 정보를 한 번 더 가공합니다. 층과 층 사이에는 ReLU activation 존재.

⇒ kernel size가 1인 2개의 convolution 적용하는 것.

단순하지만 표현력을 높여주는 역할을 합니다.

3.5 임베딩과 소프트맥스

단어를 신경망이 다룰 수 있는 숫자 벡터로 바꾸는 임베딩 과정, 그리고 최종적으로 "다음 단어가 무엇일 확률이 높은가"를 계산하는 소프트맥스 과정입니다. **입력/출력 임베딩과 최종 출력층이 같은 가중치를 공유한다는 점도 특징입니다.**

3.6 위치 인코딩(Positional Encoding)

여기서 중요한 문제가 하나 생깁니다: 어텐션은 모든 단어를 "동시에" 보기 때문에, RNN과 달리 단어의 순서 정보가 자연스럽게 들어있지 않습니다. "나는 너를 좋아해"와 "너는 나를 좋아해"를 어텐션만으로는 구분하지 못할 수 있다는 뜻이죠.

이를 해결하기 위해, 각 단어의 위치마다 ****sin/cos 함수로 만든 고유한 숫자 패턴(위치 인코딩)****을 단어 임베딩에 더해줍니다. 이렇게 하면 모델이 "이 단어는 문장의 몇 번째쯤에 있구나"를 알 수 있게 됩니다.

4. 왜 굳이 셀프 어텐션인가

논문은 셀프 어텐션을 기존 RNN·CNN과 세 가지 기준으로 비교합니다.

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

기준	RNN(2행)	CNN(3행)	셀프 어텐션(1행)
층 하나의 계산량 (complexity)	문장 길이에 비례	커널 크기에 비례	문장 길이의 제곱에 비례
병렬 처리 가능 여부	불가능 (순서대로 만)	가능	가능
두 단어 사이 최대 거리(정보가 도달하는 데 걸리는 단계)	문장 길이만큼	층을 여러 개 쌓아야 함	단 1단계 (모든 단어가 서로 바로 연결됨) $O(1)$

가장 중요한 차이는 마지막 줄입니다. 셀프 어텐션은 문장이 아무리 길어도 **어떤 두 단어든 단 한 번의 계산으로 직접 연결**됩니다. 이 덕분에 장거리 의존성을 학습하기 훨씬 쉽고, 병렬 계산도 가능해 학습 속도가 크게 빨라집니다. (다만 문장이 아주 길면 계산량이 제곱으로 늘어난다는 단점은 있습니다.)

5. 학습은 어떻게 했나

- **데이터:** WMT 2014 영어-독일어(약 450만 문장 쌍), 영어-프랑스어(약 3,600만 문장 쌍) 번역 데이터
- **하드웨어:** Nvidia P100 GPU 8개로 학습. 기본(base) 모델은 12시간, 큰(big) 모델은 3.5일 소요 — 당시 경쟁 모델들에 비해 훨씬 빠른 학습 시간
- **최적화:** Adam 옵티마이저 사용, 학습 초반엔 학습률을 점점 올리다가(warm-up) 이후 서서히 낮추는 스케줄 적용

$$lrate = d_{model}^{-0.5} \cdot \min(step.num^{-0.5}, step.num \cdot warmup.steps^{-1.5})$$

- **과적합 방지 기법:** 잔차 드롭아웃(Residual Dropout), 라벨 스무딩(Label Smoothing) 사용

6. 결과: 얼마나 잘했나

번역 품질은 **BLEU 점수**(높을수록 좋음, 사람 번역과 얼마나 비슷한지 측정)로 평가합니다.

작업	기존 최고 성능	Transformer (base)	Transformer (big)
영어 → 독일어	26.30	27.3	28.4 (신기록)
영어 → 프랑스어	41.29	38.1	41.8 (신기록)

특히 주목할 점은, Transformer가 **기존 최고 모델들보다 훨씬 적은 계산 비용**으로 이 성적을 냈다는 것입니다. 추가로 번역 외의 다른 작업(영어 구문 분석, constituency parsing)에도 적용해봤는데 여기서도 좋은 성능을 보여, "이 구조가 번역에만 특화된 게 아니라 범용적으로 잘 통한다"는 걸 보여줬습니다.

7. 결론 & 이 논문이 왜 중요한가

이 논문의 결론은 단순합니다: *****순환(recurrence)도, 합성곱(convolution)도 필요 없다. 어텐션만으로 충분하다*****는 것이 제목 그대로의 메시지입니다.

Transformer는 발표 당시에는 "번역을 더 빠르고 정확하게 하는 모델" 정도로 여겨졌지만, 이후 이 구조는 딥러닝 역사를 바꿔놓았습니다.

- **BERT(2018):** Transformer의 인코더 부분만 활용해 문장 이해에 특화
- **GPT 시리즈(2018~):** Transformer의 디코더 부분만 활용해 문장 생성에 특화
- 오늘날 ChatGPT, Claude를 포함한 거의 모든 대형 언어모델(LLM)이 이 Transformer 구조를 기본 골격으로 삼고 있습니다.

즉, "번역 논문 하나"로 시작했지만 결과적으로 **오늘날 AI 붐 전체의 기술적 토대**가 된 논문이라고 볼 수 있습니다.



핵심 용어 한눈에 정리

- **Attention(어텐션):** 입력의 각 부분이 서로 얼마나 관련 있는지 계산해서 가중치를 두어 정보를 모으는 방법
- **Self-Attention(셀프 어텐션):** 한 문장 안에서 단어들끼리 서로 어텐션을 적용하는 것

- **Query / Key / Value:** 어텐션 계산에 쓰이는 세 가지 벡터 (질문 / 색인표 / 실제 내용)
- **Multi-Head Attention:** 어텐션을 여러 개(8개)의 독립된 "시선"으로 나눠서 동시에 계산
- **Positional Encoding:** 순서 정보가 없는 어텐션 구조에 단어의 위치 정보를 더해주는 장치
- **BLEU Score:** 기계 번역 품질을 사람 번역과 비교해 수치화한 평가 지표